# PYTHON ACTIVITY

S.Shiney

CSD-1

## CASESTUDIES :41-50

### CASE 41: Nested List Classroom Occupancy Analysis

### 1.*Problem statement:*

Represent multiple classrooms and seating positions as nested lists; allocate students, preserve reserved seats, detect duplicate assignments, and calculate vacant capacity.

### 2.*Objectives:*

1. Understand nested lists in Python.

2.Allocate students to available seats.

3. Detect duplicate student assignments.

4. Calculate the remaining seating capacity.

### 3.Assumptions:

1. There are two classrooms with four seats each.

2. - represents a vacant seat, and R represents a reserved seat.

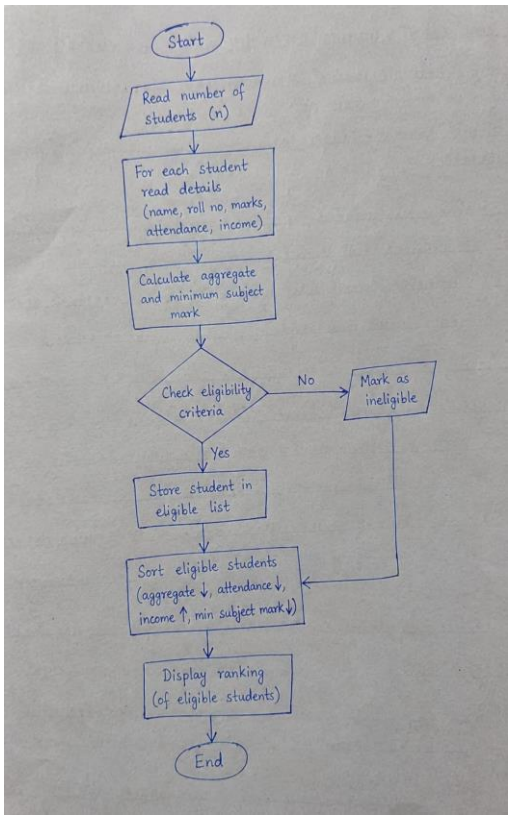
3.Each student can occupy only one seat.

4. Reserved seats cannot be allocated to students.

### 4.Algorithm:

1. **Start**
2. Create a nested list representing classrooms and seats.
3. Store student names in a list.
4. Check each student name for duplicate assignments.
5. Reject duplicate names and search for a vacant seat for other students.
6. Assign students to vacant seats while preserving reserved seats.
7. Count the remaining vacant seats.
8. Display the seating arrangement, assigned students, and vacant seats.
9. **Stop**

## 5.Flowchart:




## 6.pseudo code:

**Pseudocode: Nested List Classroom Occupancy Analysis**

1. START
2. Create a nested list to represent classrooms and their seats.
3. Input student allocations and reserved seats.
4. Initialize the occupied seat count to 0.
5. Initialize an empty list to store assigned student IDs.
6. FOR each classroom in the nested list:
    - FOR each seat in the classroom:
        - IF the seat is reserved:
            - Preserve the reserved seat.
        - ELSE IF the seat contains a student ID:
            - Check whether the student ID already exists in the assigned student list.
            - IF the student ID already exists:

        - Display "Duplicate student assignment".
    - ELSE:
        - Add the student ID to the assigned student list.
        - Increase the occupied seat count by 1.
7. Calculate the total number of seats.
8. Calculate vacant capacity by subtracting occupied and reserved seats from the total seats.
9. Display duplicate assignments, occupied seats, reserved seats, and vacant capacity.
10. STOP

## 7.Python source code:

```
# Nested List Classroom Occupancy Analysis
classrooms = [
    ["S101", "S102", None, "S104"],
    ["S105", "S106", "S101", None],
    ["Reserved", "S108", None, "S110"]
]
reserved_seats = 1
assigned_students = []
duplicates = []
occupied_seats = 0
total_seats = 0
for classroom in classrooms:
    for seat in classroom:
        total_seats += 1

        if seat == "Reserved":
            continue
```

```
        elif seat is not None:
            if seat in assigned_students:
                duplicates.append(seat)
            else:
                assigned_students.append(seat)
                occupied_seats += 1
vacant_seats = total_seats - occupied_seats - reserved_seats
print("CLASSROOM OCCUPANCY REPORT")
print("--------------------------")
print("Total Seats:", total_seats)
print("Occupied Seats:", occupied_seats)
print("Reserved Seats:", reserved_seats)
print("Vacant Seats:", vacant_seats)
print("Duplicate Student Assignments:", duplicates)
```

## 8.Test case:

**Test Case 1:** A student ID is repeated, so the program identifies a duplicate assignment.

**Test Case 2:** All student IDs are unique, and one seat is vacant.

**Test Case 3:** A duplicate student assignment is detected, and reserved seats are preserved.

**Test Case 4:** No students are assigned; the program calculates the vacant capacity.

**Test Case 5:** All seats are occupied, with no reserved seats or duplicate assignments.

## 9.Code and Output Statements

```
# Nested List Classroom Occupancy Analysis
classrooms = [
    ["S101", "S102", None, "S104"],
    ["S105", "S106", "S101", None],
    ["Reserved", "S108", None, "S110"]
]
reserved_seats = 1
assigned_students = []
duplicates = []
occupied_seats = 0
total_seats = 0
for classroom in classrooms:
    for seat in classroom:
        total_seats += 1
        if seat == "Reserved":
            continue
        elif seat is not None:
            if seat in assigned_students:
                duplicates.append(seat)
            else:
                assigned_students.append(seat)
                occupied_seats += 1
vacant_seats = total_seats - occupied_seats - reserved_seats
print("CLASSROOM OCCUPANCY REPORT")
print("--------------------------")
print("Total Seats:", total_seats)
print("Occupied Seats:", occupied_seats)
print("Reserved Seats:", reserved_seats)
print("Vacant Seats:", vacant_seats)
print("Duplicate Student Assignments:", duplicates)
```

OUTPUT:

```
CLASSROOM OCCUPANCY REPORT
--------------------------
Total Seats: 12
Occupied Seats: 7
Reserved Seats: 1
Vacant Seats: 4
Duplicate Student Assignments: ['S101']
```

## 10.CONCLUSION:

Thus, the Nested List Classroom Occupancy Analysis program was successfully implemented using Python. It uses nested lists to represent classroom seats,

identifies duplicate student assignments, preserves reserved seats, and calculates occupied and vacant seats. This program helps manage classroom seating efficiently and accurately.

# CASE 42: Student Marks Matrix Analysis

## 1.PROBLEM STATEMENT:

Student Marks Matrix Analysis: Analyse a matrix of student marks across subjects, calculate row-wise and column-wise statistics, identify top performers, and detect subjects with low average performance.

## 2.OBJECTIVES:

- To store student marks in a nested list.
- To calculate each student's total and average marks.
- To calculate each subject's average marks.
- To identify the top-performing student.
- To identify subjects with low average marks.

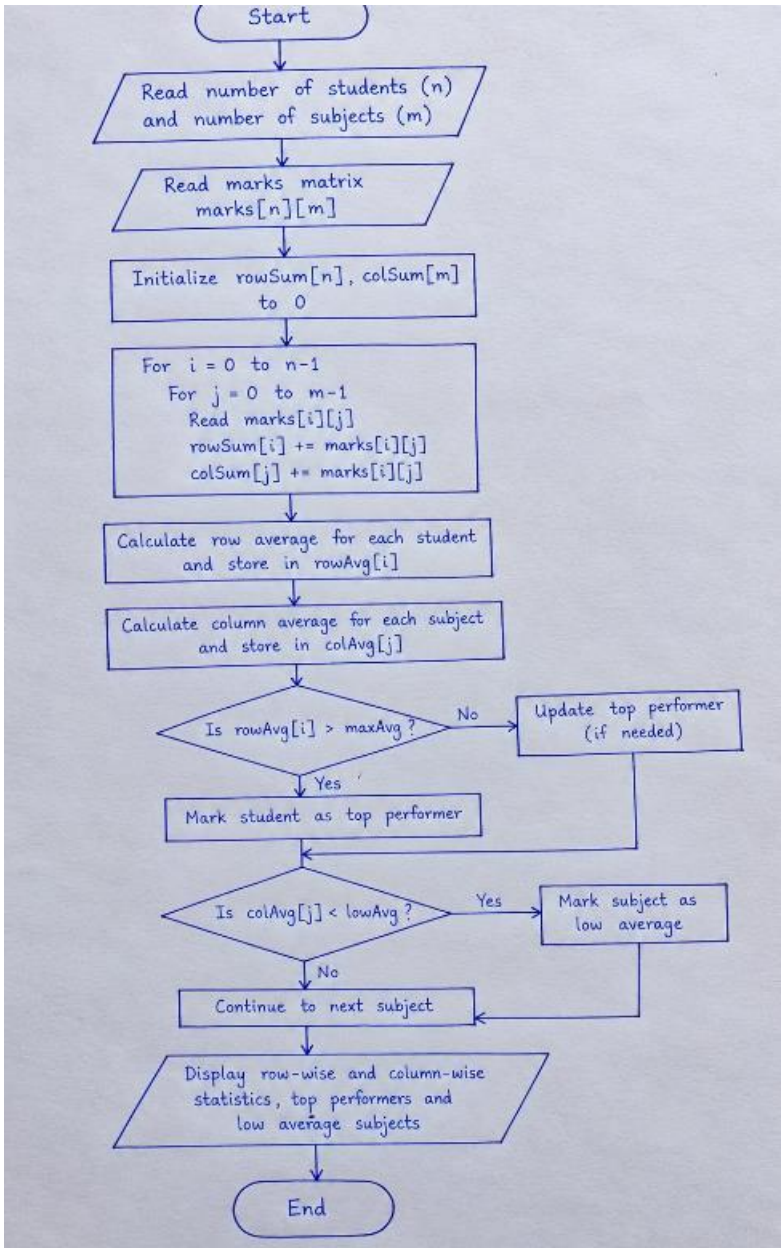
## 3.ASSUMPTIONS:

- There are 3 students and 3 subjects.
- Each subject carries 100 marks.
- A subject average below 50 is considered low.
- Student names and marks are entered correctly.

## 4.ALGORITHM:

1. Start.
2. Create a nested list containing student marks.
3. Calculate the total and average marks of each student.
4. Calculate the average marks of each subject.
5. Find the student with the highest total marks.
6. Check whether any subject average is below 50.
7. Display student totals, student averages, subject averages, the top performer, and low-performing subjects.
8. Stop.

## 5.FLOW CHART:




## 6.PSEUDO CODE:

1. START
2. Input the student marks matrix.
3. Calculate each student's total and average marks.
4. Calculate each subject's average marks.
5. Identify the student with the highest total marks.
6. Check whether any subject average is below the specified limit.

7. Display student averages, subject averages, top performer, and low-performing subjects.
8. STOP

## 7.PYTHON SOURCE CODE:

```
# Student Marks Matrix Analysis


students = ["Student 1", "Student 2", "Student 3"]

subjects = ["Python", "Maths", "English"]


marks = [
    [85, 90, 80],
    [70, 75, 65],
    [95, 92, 88]
]


# Calculate each student's total and average

totals = []

print("STUDENT MARKS REPORT")

print("--------------------")

for i in range(len(students)):
    total = sum(marks[i])
    average = total / len(subjects)
    totals.append(total)
    print(students[i], "Total:", total, "Average:",
round(average, 2))

# Calculate each subject's average

print("\nSUBJECT AVERAGES")

for j in range(len(subjects)):
    total = 0
    for i in range(len(students)):
```

```
        total += marks[i][j]
    average = total / len(students)
    print(subjects[j], "Average:", round(average, 2))
    if average < 70:
        print("Low-performing subject:", subjects[j])
# Identify the top performer
top_index = totals.index(max(totals))
print("\nTop Performer:", students[top_index])
print("Highest Total Marks:", totals[top_index])
```

## 8.TEST CASE:

**Test Case 1:** When the program receives marks of three students in three subjects, it calculates each student's total and average marks and displays the subject-wise averages. The program identifies Student 3 as the top performer with 275 total marks.

**Test Case 2:** When a subject's average marks are below 70, the program displays that subject as a low-performing subject. If all subject averages are 70 or above, no low-performing subject is displayed.

**Test Case 3:** When students have different marks in each subject, the program calculates the correct row-wise and column-wise averages and identifies the student with the highest total marks.

**Test Case 4:** When all students score equal total marks, the program identifies the first student with the highest total as the top performer.

## 9.CODE AND OUTPUT SCREENSHOT:

```
# Student Marks Matrix Analysis
students = ["Student 1", "Student 2", "Student 3"]
subjects = ["Python", "Maths", "English"]
marks = [
    [85, 90, 80],
    [70, 75, 65],
    [95, 92, 88]
]
# Calculate each student's total and average
totals = []
print("STUDENT MARKS REPORT")
print("--------------------")
for i in range(len(students)):
    total = sum(marks[i])
    average = total / len(subjects)
    totals.append(total)
    print(students[i], "Total:", total, "Average:", round(average, 2))
# Calculate each subject's average
print("\nSUBJECT AVERAGES")
for j in range(len(subjects)):
    total = 0
    for i in range(len(students)):
        total += marks[i][j]
    average = total / len(students)
    print(subjects[j], "Average:", round(average, 2))
    if average < 70:
        print("Low-performing subject:", subjects[j])
# Identify the top performer
top_index = totals.index(max(totals))
print("\nTop Performer:", students[top_index])
```

OUTPUT:

```
STUDENT MARKS REPORT
--------------------
Student 1 Total: 255 Average: 85.0
Student 2 Total: 210 Average: 70.0
Student 3 Total: 275 Average: 91.67

SUBJECT AVERAGES
Python Average: 83.33
Maths Average: 85.67
English Average: 77.67

Top Performer: Student 3
Highest Total Marks: 275
```

## 10.CONCLUSION:

Thus, the Student Marks Matrix Analysis program was successfully implemented using Python. It calculates each student's total and average marks, finds subject-wise averages, identifies the top-performing student, and detects subjects with low average marks. This program helps analyze student performance accurately and efficiently.

## CASE43: Multi-Branch Sales Matrix Comparison

## 1.PROBLEM STATEMENT:

Store product sales across branches and days in nested lists, calculate product totals, compare branches, and identify the best-performing product for each branch.

## 2.OBJECTIVES:

To develop a Python program using nested lists to store product sales data across multiple branches and days, calculate total sales for each product, compare sales among branches, and identify the best-performing product in each branch.

## 3.ASSUMPTIONS:

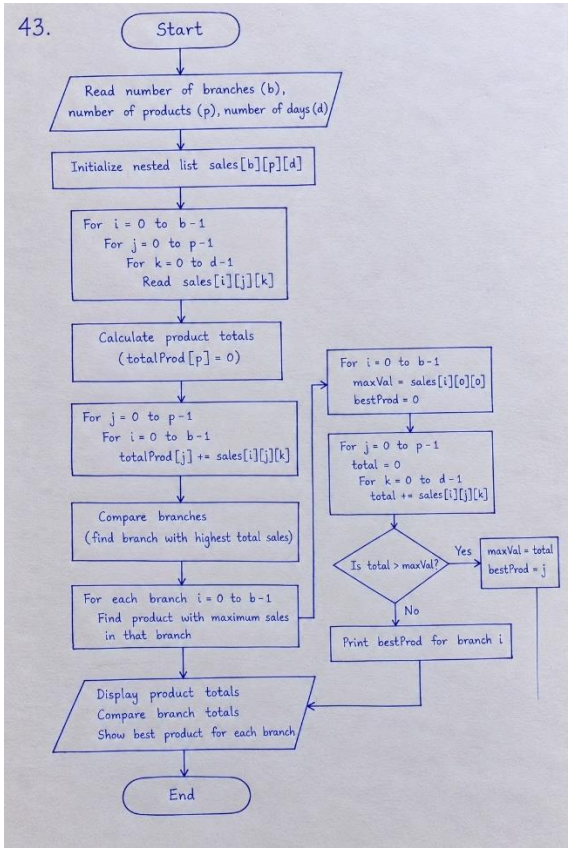
1. Sales data is available for multiple products across different branches and days.
2. Nested lists are used to store the product sales data.
3. All sales values are non-negative numbers.
4. Each branch records sales for the same set of products.
5. Total product sales are calculated by adding daily sales.
6. The branch with the highest total sales is considered the best-performing branch.
7. The product with the highest sales in each branch is considered the best-performing product.

## 4.ALGORITHM:

1. Start.
2. Create a nested list to store product sales across branches and days.
3. Calculate the total sales of each product by adding its daily sales.
4. Calculate the total sales of each branch.
5. Compare the total sales of all branches.
6. Identify the best-performing branch with the highest total sales.
7. Identify the best-performing product in each branch.

8. Display product totals, branch-wise sales, and the best-performing products.
9. Stop.

## 5.FLOWCHART:




## 6.PSEUDO CODE:

1. START
2. Input product sales data for all branches and days.
3. Calculate the total sales of each product.
4. Calculate the total sales of each branch.
5. Compare the total sales of all branches.
6. Identify the branch with the highest total sales.
7. Find the best-performing product in each branch.
8. Display product totals, branch totals, and the best-performing products.
9. STOP

## 7.PYTHON SOURCE CODE:

```
# Multi-Branch Sales Matrix Comparison
```

```
branches = ["Branch 1", "Branch 2", "Branch 3"]
products = ["Laptop", "Mobile", "Tablet"]

# Sales data: each branch contains daily sales for each product
sales = [
    [[10, 12, 8], [15, 10, 5], [6, 8, 4]],
    [[12, 10, 9], [10, 14, 6], [8, 7, 5]],
    [[15, 12, 10], [8, 9, 7], [5, 6, 4]]
]

branch_totals = []

for i in range(len(branches)):
    print("\n", branches[i])
    total_branch = 0

    for j in range(len(products)):
        total_product = sum(sales[i][j])
        total_branch += total_product

        print(products[j], "Total Sales:", total_product)

    branch_totals.append(total_branch)
    print("Total Branch Sales:", total_branch)

    best_product_index = 0

    for j in range(1, len(products)):
        if sum(sales[i][j]) > sum(sales[i][best_product_index]):
```

```
            best_product_index = j


    print("Best Performing Product:", products[best_product_index])


# Compare branches
best_branch_index = branch_totals.index(max(branch_totals))


print("\nBest Performing Branch:", branches[best_branch_index])
print("Highest Branch Sales:", branch_totals[best_branch_index])
```

## 8.TEST CASE:

**Test Case 1:** When sales data is provided for three branches and three products, the program calculates the total sales of each product and the total sales of each branch. It identifies Branch 2 as the best-performing branch with total sales of 81 units.

**Test Case 2:** When a branch has different sales values for each product, the program adds the daily sales and identifies the product with the highest total sales in that branch.

**Test Case 3:** When all branches have different total sales, the program compares their totals and displays the branch with the highest sales.

**Test Case 4:** When a product has zero sales on one or more days, the program calculates its total sales correctly by adding the available daily sales values.

**Test Case 5:** When two products have equal highest sales in a branch, the program displays the first product with the highest total sales.

## 9.CODE AND OUTPUT SCREENSHOT:

```python
branches = ["Branch 1", "Branch 2", "Branch 3"]
products = ["Laptop", "Mobile", "Tablet"]
sales = [
    [[10, 12, 8], [15, 10, 5], [6, 8, 4]],
    [[12, 10, 9], [10, 14, 6], [8, 7, 5]],
    [[15, 12, 10], [8, 9, 7], [5, 6, 4]]
]

branch_totals = []

for i in range(len(branches)):
    print("\n", branches[i])
    total_branch = 0

    for j in range(len(products)):
        total_product = sum(sales[i][j])
        total_branch += total_product

        print(products[j], "Total Sales:", total_product)

    branch_totals.append(total_branch)
    print("Total Branch Sales:", total_branch)

    best_product_index = 0

    for j in range(1, len(products)):
        if sum(sales[i][j]) > sum(sales[i][best_product_index]):
            best_product_index = j

    print("Best Performing Product:", products[best_product_index])
best_branch_index = branch_totals.index(max(branch_totals))

print("\nBest Performing Branch:", branches[best_branch_index])
print("Highest Branch Sales:", branch_totals[best_branch_index])
```

OUTPUT:

```
 Branch 1
Laptop Total Sales: 30
Mobile Total Sales: 30
Tablet Total Sales: 18
Total Branch Sales: 78
Best Performing Product: Laptop

 Branch 2
Laptop Total Sales: 31
Mobile Total Sales: 30
Tablet Total Sales: 20
Total Branch Sales: 81
Best Performing Product: Laptop

 Branch 3
Laptop Total Sales: 37
Mobile Total Sales: 24
Tablet Total Sales: 15
Total Branch Sales: 76
Best Performing Product: Laptop

Best Performing Branch: Branch 2
Highest Branch Sales: 81
```

## 10.CONCLUSION:

Thus, the Multi-Branch Sales Matrix Comparison program was successfully implemented using Python. It uses nested lists to store sales data, calculates product-wise and branch-wise total sales, compares different branches, and identifies the best-performing product in each branch. This program helps businesses analyze sales performance and make better decisions.

## CASE 44: Hospital Ward Occupancy Monitoring

## 1.PROBLEM STATEMENT:

Analyse a nested list of bed occupancy counts by ward and day, calculate utilization percentages, detect capacity violations, and identify peak-demand periods.

## 2.OBJECTIVES:

1. To store daily hospital ward occupancy data using nested lists.
2. To calculate the utilization percentage for each ward.
3. To detect occupancy exceeding the ward's bed capacity.
4. To identify the day with the highest total occupancy.

## 3.ASSUMPTIONS:

1. Each row in the nested list represents a hospital ward, and each column represents a day.
2. The bed capacity of each ward is provided separately.
3. Occupancy values represent the number of occupied beds in each ward on each day.
4. Occupancy values are non-negative integers.
5. The occupancy percentage is calculated by dividing occupied beds by ward capacity and multiplying by 100.
6. A capacity violation occurs when occupied beds exceed the ward's total capacity.
7. The day with the highest combined occupancy across all wards is considered the peak-demand day.

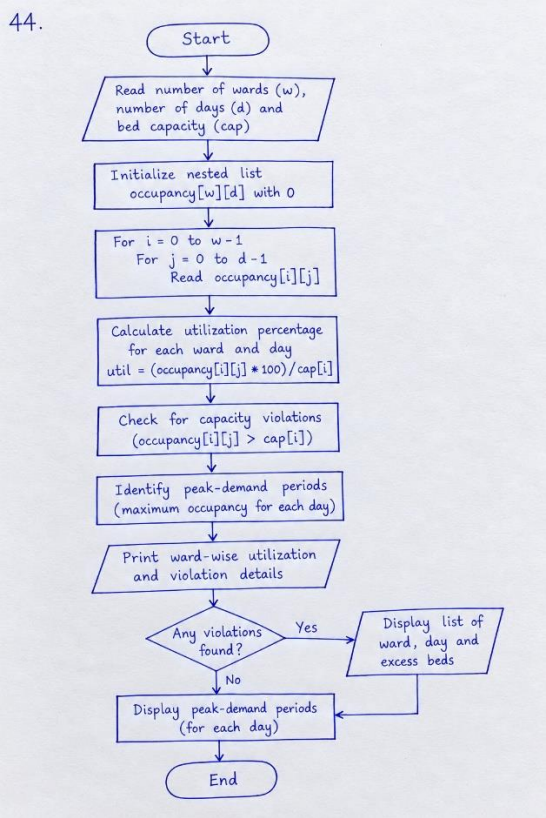
## 4.ALGORITHM:

1. Start.
2. Create lists for ward names, ward capacities, days, and occupancy data.
3. For each ward and each day, calculate the utilization percentage using the formula:

   **Utilization = (Occupied Beds / Ward Capacity) × 100**

4. Check whether occupied beds exceed the ward capacity.
5. If occupancy exceeds capacity, display a capacity violation message.
6. Calculate the total occupied beds for each day.
7. Identify the day with the highest total occupancy.
8. Display the utilization percentages, capacity violations, daily totals, and peak-demand day.
9. Stop.

## 5.FLOWCHART:




## 6.PSEUDO CODE:

```
BEGIN
   FOR each ward
      FOR each day
         utilization = (occupied beds / capacity) * 100
         IF occupied beds > capacity THEN
            Display "Capacity Violation"
         ENDIF
```

```
        ENDFOR
    ENDFOR

    Calculate total occupied beds for each day
    Find the day with maximum occupancy
    Display utilization percentages
    Display capacity violations
    Display peak-demand day
END
```

## 7.PYTHON SOURCE CODE:

```
wards = ["Ward A", "Ward B", "Ward C"]
capacities = [30, 20, 25]
days = ["Mon", "Tue", "Wed"]

occupancy = [
    [25, 29, 31],
    [15, 18, 17],
    [20, 22, 24]
]

daily_totals = [0] * len(days)

for i in range(len(wards)):
    for j in range(len(days)):
        occupied = occupancy[i][j]
        daily_totals[j] += occupied

        utilization = occupied / capacities[i] * 100
```

```
        print(
            wards[i], days[j],
            "Utilization:", round(utilization, 1), "%"
        )

        if occupied > capacities[i]:
            print("CAPACITY VIOLATION:", wards[i], days[j])

peak_index = daily_totals.index(max(daily_totals))

print("Daily Totals:", daily_totals)
print("Peak-Demand Day:", days[peak_index])
```

## 8.TESTCASES:

**Test Case 1: Normal Occupancy**
**Input:** Ward A has 25 occupied beds out of 30 beds.
**Expected Output:** Utilization is 83.3%. No capacity violation is reported.

**Test Case 2: Occupancy Exceeds Capacity**
**Input:** Ward A has 31 occupied beds out of 30 beds.
**Expected Output:** Utilization is 103.3%, and a capacity violation is reported.

**Test Case 3: Occupancy Equals Capacity**
**Input:** Ward B has 20 occupied beds out of 20 beds.
**Expected Output:** Utilization is 100%. No capacity violation is reported.

**Test Case 4: Peak-Demand Day**
**Input:** Daily total occupancies are Monday – 60, Tuesday – 69, Wednesday – 72.
**Expected Output:** Wednesday is identified as the peak-demand day.

## 9.CODE AND OUTPUT SCREENSHOTS:

```python
wards = ["Ward A", "Ward B", "Ward C"]
capacities = [30, 20, 25]
days = ["Mon", "Tue", "Wed"]

occupancy = [
    [25, 29, 31],
    [15, 18, 17],
    [20, 22, 24]
]

daily_totals = [0] * len(days)

for i in range(len(wards)):
    for j in range(len(days)):
        occupied = occupancy[i][j]
        daily_totals[j] += occupied

        utilization = occupied / capacities[i] * 100

        print(
            wards[i], days[j],
            "Utilization:", round(utilization, 1), "%"
        )

        if occupied > capacities[i]:
            print("CAPACITY VIOLATION:", wards[i], days[j])

peak_index = daily_totals.index(max(daily_totals))

print("Daily Totals:", daily_totals)
print("Peak-Demand Day:", days[peak_index])
```

Output:

```
Ward A Mon Utilization: 83.3 %
Ward A Tue Utilization: 96.7 %
Ward A Wed Utilization: 103.3 %
CAPACITY VIOLATION: Ward A Wed
Ward B Mon Utilization: 75.0 %
Ward B Tue Utilization: 90.0 %
Ward B Wed Utilization: 85.0 %
Ward C Mon Utilization: 80.0 %
Ward C Tue Utilization: 88.0 %
Ward C Wed Utilization: 96.0 %
Daily Totals: [60, 69, 72]
Peak-Demand Day: Wed
```

## 10.CONCLUSION:

The Hospital Ward Occupancy Monitoring program calculates ward utilization percentages, detects capacity violations, and identifies the day with the highest total occupancy. It helps monitor hospital bed usage and supports efficient hospital resource management.

## CASE 45: Weekly Electricity Consumption Matrix

## 1.PROBLEM STATEMENT:

Process appliance-wise electricity readings across seven days, calculate daily and appliance totals, and identify consumption exceeding specified limits.

## 2.OBJECTIVES:

1. To represent appliance readings using a nested list.

2. To calculate daily and appliance-wise electricity consumption totals.
3. To identify the appliance with the highest weekly consumption.
4. To flag daily electricity consumption that exceeds the specified limit.

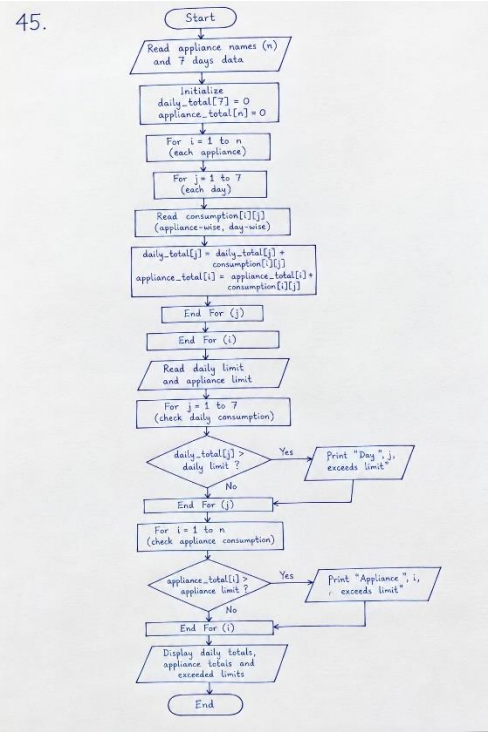
## 3.ASSUMPTIONS:

1. Each row represents an electrical appliance, and each column represents one of the seven days.
2. Electricity consumption readings are measured in kilowatt-hours (kWh).
3. The daily electricity consumption limit for all listed appliances combined is **10 kWh**.
4. Each appliance has seven daily consumption readings.
5. All consumption readings are non-negative value

## 4.ALGORITHM:

1. Start.
2. Create a list of appliance names and their seven-day electricity consumption readings.
3. Calculate and display the weekly consumption total for each appliance.
4. Calculate the total electricity consumption for each day.
5. Compare each daily total with the specified limit.
6. If the daily total exceeds the limit, display a warning message.
7. Identify the appliance with the highest weekly consumption.
8. Display all results.
9. Stop.

## 5.FLOWCHART:




## 6.PSEUDO CODE:

**PSEUDOCODE – WEEKLY ELECTRICITY CONSUMPTION MATRIX**

BEGIN

Create a list of appliance names

Store seven-day electricity readings for each appliance

Set the daily consumption limit

FOR each appliance

Calculate weekly consumption total

Display appliance name and weekly total

ENDFOR

FOR each day from 1 to 7

Calculate total electricity consumption for that day

Display daily total

```
    IF daily total > daily limit THEN
        Display "Warning: daily limit exceeded"
    ENDIF
  ENDFOR

  Find the appliance with the highest weekly consumption
  Display the highest-consuming appliance

END
```

## 7.PYTHON SOURCE CODE:

```
appliances = ["Fan", "Light", "Computer"]

# Seven readings per appliance, in kWh
usage = [
    [1.2, 1.1, 1.3, 1.2, 1.1, 1.4, 1.0],
    [0.8, 0.7, 0.8, 0.9, 0.7, 0.8, 0.9],
    [2.5, 2.8, 3.0, 2.6, 2.9, 3.2, 2.7]
]

daily_limit = 5.0
appliance_totals = []

# Calculate weekly consumption for each appliance
for i in range(len(appliances)):
    total = sum(usage[i])
    appliance_totals.append(total)
    print(appliances[i], "weekly consumption:",
          round(total, 2), "kWh")
```

```
# Calculate daily consumption
for day in range(7):
    daily_total = 0

    for appliance in range(len(appliances)):
        daily_total += usage[appliance][day]

    print("Day", day + 1, "total:",
          round(daily_total, 2), "kWh")

    if daily_total > daily_limit:
        print("Warning: daily limit exceeded")

# Identify the appliance with the highest weekly consumption
highest = appliance_totals.index(max(appliance_totals))

print("Highest-consuming appliance:", appliances[highest])
```

## 8.TESTCASES:

**TEST CASES – WEEKLY ELECTRICITY CONSUMPTION MATRIX**

**Test Case 1:** When the daily electricity consumption is less than 5.0 kWh, the program displays the daily total without showing a warning.

**Test Case 2:** When the daily electricity consumption equals 5.0 kWh, the program does not display a warning because the consumption has not exceeded the limit.

**Test Case 3:** When the daily electricity consumption exceeds 5.0 kWh, the program displays the warning message "Warning: daily limit exceeded".

**Test Case 4:** When different appliances have different weekly consumption values, the program calculates and displays the weekly consumption of each appliance.

**Test Case 5:** When one appliance has the highest weekly consumption, the program identifies and displays the name of that appliance.

## 9.CODE AND OUTPUT SCREENSHOTS:

```
appliances = ["Fan", "Light", "Computer"]
usage = [
    [1.2, 1.1, 1.3, 1.2, 1.1, 1.4, 1.0],
    [0.8, 0.7, 0.8, 0.9, 0.7, 0.8, 0.9],
    [2.5, 2.8, 3.0, 2.6, 2.9, 3.2, 2.7]
]

daily_limit = 5.0
appliance_totals = []
for i in range(len(appliances)):
    total = sum(usage[i])
    appliance_totals.append(total)
    print(appliances[i], "weekly consumption:",
          round(total, 2), "kWh")
for day in range(7):
    daily_total = 0

    for appliance in range(len(appliances)):
        daily_total += usage[appliance][day]

    print("Day", day + 1, "total:",
          round(daily_total, 2), "kWh")

    if daily_total > daily_limit:
        print("Warning: daily limit exceeded")
highest = appliance_totals.index(max(appliance_totals))

print("Highest-consuming appliance:", appliances[highest])
```

OUTPUT:

```
Fan weekly consumption: 8.3 kWh
Light weekly consumption: 5.6 kWh
Computer weekly consumption: 19.7 kWh
Day 1 total: 4.5 kWh
Day 2 total: 4.6 kWh
Day 3 total: 5.1 kWh
Warning: daily limit exceeded
Day 4 total: 4.7 kWh
Day 5 total: 4.7 kWh
Day 6 total: 5.4 kWh
Warning: daily limit exceeded
Day 7 total: 4.6 kWh
Highest-consuming appliance: Computer
```

## 10.CONCLUSION:

The Weekly Electricity Consumption Matrix provides an effective way to calculate daily and appliance-wise electricity consumption using a nested list in Python. The program calculates weekly totals for each appliance, identifies the appliance with the highest consumption, and displays warnings when daily electricity consumption exceeds the specified limit. Thus, it helps monitor electricity usage and identify excessive consumption.

## CASE 46: Inventory Matrix Reconciliation

## 1.PROBLEM STATEMENT:

Compare product quantities across warehouses using nested lists, identify shortages and surpluses, and calculate the transfers required to meet specified stock targets.

## 2.OBJECTIVES:

1. Store warehouse stock in a nested list.
2. Calculate total stock for each product.
3. Compare actual stock with target stock.
4. Report shortages and surpluses.

## 3.ASSUMPTIONS:

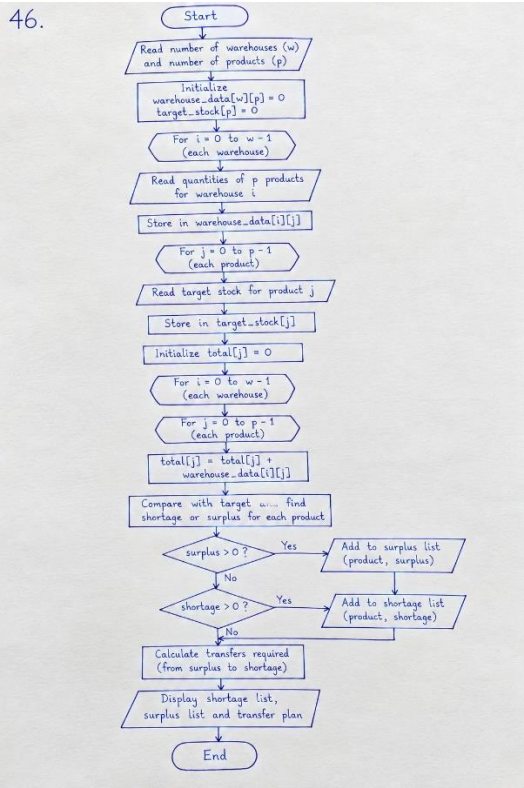
1. Rows represent warehouses, and columns represent products.
2. Targets are the required total quantities across all warehouses.
3. Only total stock is compared with the target; transport costs are not considered.

## 4.ALGORITHM:

**ALGORITHM – INVENTORY MATRIX RECONCILIATION**

1. **Start.**
2. Create warehouse names, product names, and the stock matrix.
3. Calculate the total stock of each product across all warehouses.
4. Compare each product's total stock with its target quantity.
5. Calculate the difference between total stock and target stock.
6. If the difference is negative, report a shortage.
7. If the difference is positive, report a surplus.
8. If the difference is zero, display that the stock exactly meets the target.
9. Display the results for all products.
10. **Stop**

## 5.FLOWCHART:




## 6.PSEUDO CODE:

```
BEGIN
   FOR each product column
      total_stock = sum quantities across warehouses
      difference = total_stock - target

      IF difference < 0 THEN
         Report shortage
      ELSE IF difference > 0 THEN
         Report surplus
      ELSE
         Report balanced stock
      ENDIF
   ENDFOR
END
```

## 7.PYTHON SOURCE CODE:

```
warehouses = ["Warehouse A", "Warehouse B", "Warehouse C"]
products = ["Pens", "Books", "Bags"]

stock = [
    [100, 40, 15],
    [80, 60, 20],
    [70, 50, 10]
]

targets = [300, 180, 50]

for j in range(len(products)):
    total = 0

    for i in range(len(warehouses)):
        total += stock[i][j]

    difference = total - targets[j]

    print(products[j], "total stock:", total, "target:", targets[j])

    if difference < 0:
        print("Shortage:", -difference)

    elif difference > 0:
        print("Surplus:", difference)

    else:
```

print("Stock exactly meets target")

## 8.TESTCASES:

**Test Case 1: Shortage of Stock**
When the total stock of a product is less than the target quantity, the program displays the shortage amount.

**Test Case 2: Surplus of Stock**
When the total stock of a product is greater than the target quantity, the program displays the surplus amount.

**Test Case 3: Stock Meets Target**
When the total stock of a product equals the target quantity, the program displays a message stating that the stock exactly meets the target.

## 9.CODE AND OUTPUT SCREENSHOTS:

```
warehouses = ["Warehouse A", "Warehouse B", "Warehouse C"]
products = ["Pens", "Books", "Bags"]

stock = [
    [100, 40, 15],
    [80, 60, 20],
    [70, 50, 10]
]

targets = [300, 180, 50]

for j in range(len(products)):
    total = 0

    for i in range(len(warehouses)):
        total += stock[i][j]

    difference = total - targets[j]

    print(products[j], "total stock:", total, "target:", targets[j])

    if difference < 0:
        print("Shortage:", -difference)

    elif difference > 0:
        print("Surplus:", difference)

    else:
        print("Stock exactly meets target")
```

OUTPUT:

```
---------- RESTART: C:/Use
Pens total stock: 250 target: 300
Shortage: 50
Books total stock: 150 target: 180
Shortage: 30
Bags total stock: 45 target: 50
Shortage: 5
```

## 10.CONCLUSION:

The Inventory Matrix Reconciliation program uses nested lists to manage stock quantities across multiple warehouses. It calculates the total stock of each product and compares it with the target quantity. The program identifies shortages, surpluses, and products that exactly meet their targets. Thus, it helps in monitoring inventory levels and making better stock management decisions.

## CASE 47 : Sports Tournament Ranking System

## 1.PROBLEM STATEMENT:

Store match results and team scores, calculate total points and tie-break values, and produce rankings under a specified competition policy.

## 2.OBSERVATIONS:

1. Team results are stored in lists.
2. Tournament points are calculated based on wins, draws, and losses.
3. Score difference is calculated using scores for and against.
4. Teams are ranked using points and score difference.

## 3.ASSUMPTIONS:

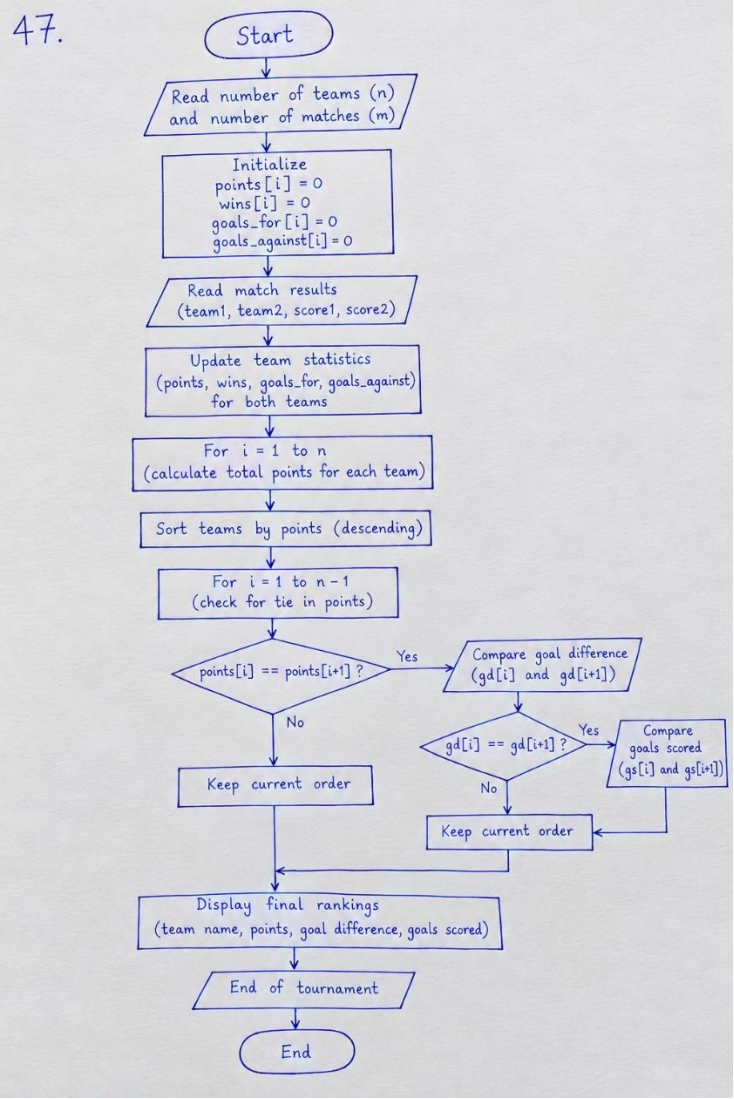
1. A win gives 3 points, a draw gives 1 point, and a loss gives 0 points.
2. Each match record is (team1, score1, team2, score2).
3. Ranking uses points first and score difference second.

## 4.ALGORITHM:

1. Start.
2. Create a list of teams and initialize points, scores for, and scores against to zero.
3. Process each match record.
4. Update the scores for and against for both teams.
5. Award 3 points to the winning team, 1 point to each team in a draw, and 0 points to the losing team.

6. Calculate the score difference for each team.
7. Sort the teams in descending order based on points and then score difference.
8. Display the final tournament rankings.
9. Stop.

## 5.FLOWCHART:




## 6.PSEUDO CODE:

BEGIN

Initialise points, scores for, and scores against to zero

FOR each match (team1, score1, team2, score2)

Update scores for and against for both teams

```
        IF score1 > score2
            Award 3 points to team1
        ELSE IF score1 < score2
            Award 3 points to team2
        ELSE
            Award 1 point to each team
        ENDIF
    ENDFOR

    Calculate score difference for every team

    Sort teams by points and then score difference in descending order

    Display the final tournament ranking
END
```

## 7.PYTHON SOURCE CODE:

```
teams = ["Tigers", "Eagles", "Lions"]

# Match format: (team1, score1, team2, score2)
matches = [
    ("Tigers", 3, "Eagles", 1),
    ("Lions", 2, "Tigers", 2),
    ("Eagles", 2, "Lions", 0)
]

# Initialize team statistics
stats = {}
```

```
for team in teams:
    stats[team] = {"points": 0, "for": 0, "against": 0}

# Process match results
for t1, s1, t2, s2 in matches:
    stats[t1]["for"] += s1
    stats[t1]["against"] += s2

    stats[t2]["for"] += s2
    stats[t2]["against"] += s1

    if s1 > s2:
        stats[t1]["points"] += 3
    elif s2 > s1:
        stats[t2]["points"] += 3
    else:
        stats[t1]["points"] += 1
        stats[t2]["points"] += 1

# Rank teams by points and score difference
ranking = sorted(
    teams,
    key=lambda team: (
        stats[team]["points"],
        stats[team]["for"] - stats[team]["against"]
    ),
    reverse=True
)
```

```
# Display tournament ranking
print("Rank  Team    Points  Difference")


for rank, team in enumerate(ranking, start=1):
    difference = stats[team]["for"] - stats[team]["against"]
    print(rank, team, stats[team]["points"], difference)
```

## 8.TESTCASES:

**Test Cases:**

Test Case 1: When a team wins a match, the winning team should receive 3 points.

Test Case 2: When a match ends in a draw, both teams should receive 1 point each.

Test Case 3: When a team loses a match, the losing team should receive 0 points.

Test Case 4: When two teams have equal points but different score differences, the team with the higher score difference should be ranked higher.

Test Case 5: After processing all match results, the program should display the teams in descending order based on points and score difference.

## 9.CODE AND OUTPUT SCREENSHOTS:



```
teams = ["Tigers", "Eagles", "Lions"]
matches = [
    ("Tigers", 3, "Eagles", 1),
    ("Lions", 2, "Tigers", 2),
    ("Eagles", 2, "Lions", 0)
]
stats = {}
for team in teams:
    stats[team] = {"points": 0, "for": 0, "against": 0}
for t1, s1, t2, s2 in matches:
    stats[t1]["for"] += s1
    stats[t1]["against"] += s2
    stats[t2]["for"] += s2
    stats[t2]["against"] += s1
    if s1 > s2:
        stats[t1]["points"] += 3
    elif s2 > s1:
        stats[t2]["points"] += 3
    else:
        stats[t1]["points"] += 1
        stats[t2]["points"] += 1
ranking = sorted(
    teams,
    key=lambda team: (
        stats[team]["points"],
        stats[team]["for"] - stats[team]["against"]
    ),
    reverse=True
)
print("Rank  Team    Points  Difference")

for rank, team in enumerate(ranking, start=1):
    difference = stats[team]["for"] - stats[team]["against"]
    print(rank, team, stats[team]["points"], difference)
```

Output:

```
========== RESTART: C:/Use
Rank  Team    Points  Difference
1 Tigers 4 2
2 Eagles 3 0
3 Lions 1 -2
```

## 10.CONCLUSION:

The Sports Tournament Ranking System successfully processes match results and calculates points for each team based on wins, draws, and losses. It calculates the score difference using scores for and against and ranks the teams according to points and score difference. Thus, the program helps determine the final tournament rankings accurately.

## CASE 48: Tuple-Based Employee Shift Conflict Detection

## 1.PROBLEM STATEMENT:

Represent employee shifts as tuples, compare time intervals, detect overlapping assignments, and identify employees with conflicting schedules.

## 2.OBJECTIVES:

1. Represent employee work shifts using tuples.
2. Compare the start and end times of shifts.
3. Detect overlapping shifts assigned to the same employee.
4. Report conflicting shift pairs.

## 3.ASSUMPTIONS:

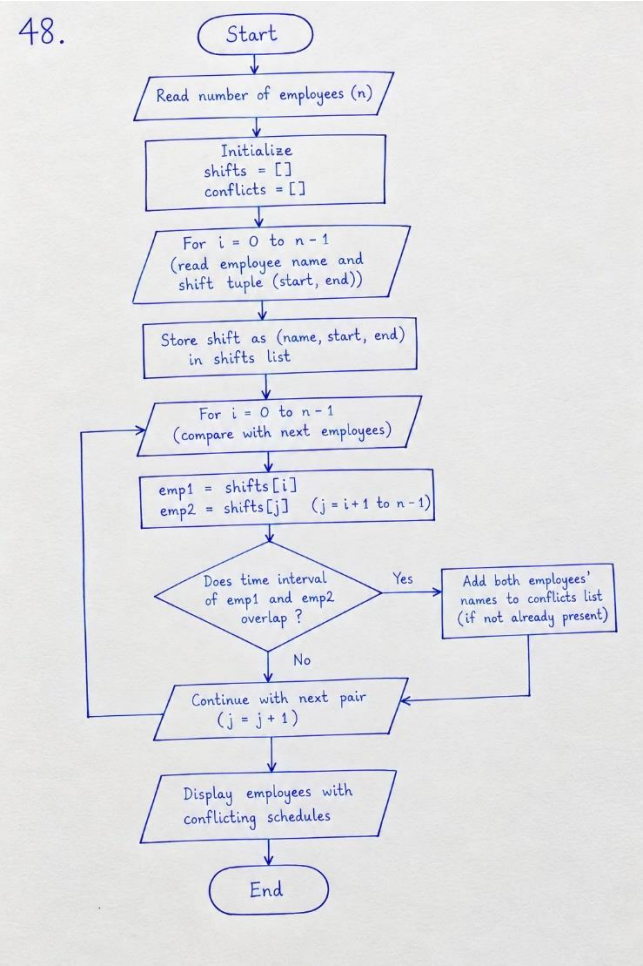
1. Times are represented as integer hours from 0 to 23.
2. A shift ends after it starts and does not cross midnight.
3. Two shifts overlap when each shift starts before the other shift ends.

## 4.ALGORITHM:

1. Start.
2. Create a list of employee shift tuples containing the employee name, start time, and end time.
3. Compare every pair of shifts.
4. Check whether both shifts belong to the same employee.

5. Check whether the shifts overlap using the condition start1 < end2 and start2 < end1.
6. If both conditions are true, display the conflicting shift pairs.
7. Repeat the process until all pairs of shifts are checked.
8. Stop.

## 5.FLOWCHART:




## 6.PSEUDO CODE:

BEGIN

Create a list of shift tuples

FOR each pair of shifts i and j, where j > i

IF employee names are the same THEN

IF start time of shift i < end time of shift j

AND start time of shift j < end time of shift i THEN

```
            Display the conflicting shift pair
        ENDIF
    ENDIF
  ENDFOR

END
```

## 7.PYTHON SOURCE CODE:

```
shifts = [
    ("Asha", 9, 12),
    ("Ravi", 10, 14),
    ("Asha", 11, 13),
    ("Asha", 13, 16),
    ("Ravi", 15, 17)
]

conflict_found = False

for i in range(len(shifts)):
    name1, start1, end1 = shifts[i]

    for j in range(i + 1, len(shifts)):
        name2, start2, end2 = shifts[j]

        if name1 == name2 and start1 < end2 and start2 < end1:
            print("Conflict:", shifts[i], "and", shifts[j])
            conflict_found = True

if not conflict_found:
    print("No shift conflicts found.")
```

## 8.TESTCASES:

Test Case 1: When the same employee, Asha, has shifts from 9 AM to 12 PM and 11 AM to 1 PM, the shifts overlap, so the program reports a conflict.

Test Case 2: When two different employees, Asha and Ravi, have overlapping shifts from 9 AM to 12 PM and 10 AM to 2 PM, no conflict is reported because they are different employees.

Test Case 3: When the same employee, Asha, has shifts from 9 AM to 12 PM and 1 PM to 4 PM, the shifts do not overlap, so no conflict is reported.

Test Case 4: When one shift ends at 12 PM and the next shift starts at 12 PM for the same employee, no conflict is reported because the shifts do not overlap.

Test Case 5: When no shifts overlap for the same employee, the program displays **"No shift conflicts found."**

## 9.CODE AND OUTPUT SCREENSHOTS:

```
shifts = [
    ("Asha", 9, 12),
    ("Ravi", 10, 14),
    ("Asha", 11, 13),
    ("Asha", 13, 16),
    ("Ravi", 15, 17)
]

conflict_found = False

for i in range(len(shifts)):
    name1, start1, end1 = shifts[i]

    for j in range(i + 1, len(shifts)):
        name2, start2, end2 = shifts[j]

        if name1 == name2 and start1 < end2 and start2 < end1:
            print("Conflict:", shifts[i], "and", shifts[j])
            conflict_found = True

if not conflict_found:
    print("No shift conflicts found.")
```

OUTPUT:

```
Conflict: ('Asha', 9, 12) and ('Asha', 11, 13)
```

## 10.CONCLUSION:

The Tuple-Based Employee Shift Conflict Detection program successfully identifies overlapping shifts assigned to the same employee. By using tuples and nested loops, the program compares shift timings and reports conflicts. It helps prevent scheduling errors and ensures proper management of employee work shifts.

## CASE 49: Tuple-Based Product Catalogue Audit

### 1.PROBLEM STATEMENT:

Store product identifiers, prices, and stock quantities in tuples; detect invalid records, calculate inventory values, and identify items requiring replenishment.

### 2.OBJECTIVES:

1. Represent product records using tuples.
2. Validate prices and stock quantities.
3. Calculate each product's inventory value.
4. Identify products below the reorder level.

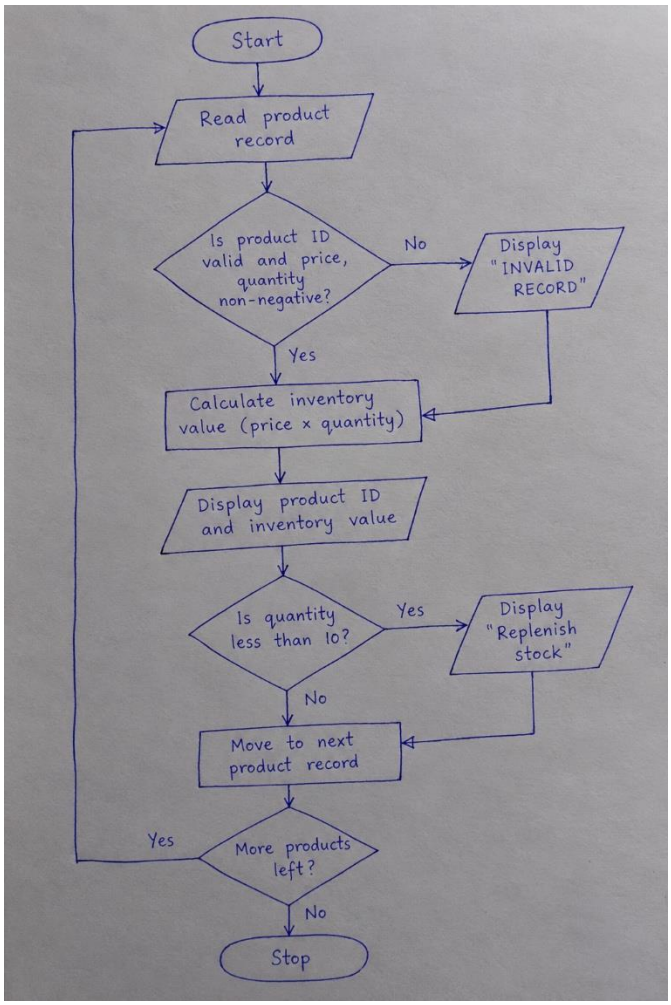
### 3.ASSUMPTIONS:

1. Each product record is represented as a tuple in the format (product_id, price, quantity).
2. The price and stock quantity must not be negative.
3. Products with fewer than **10 units** in stock require replenishment.

### 4.ALGORITHM:

1. Start.
2. Create a list of product tuples containing product ID, price, and quantity.
3. Check each product record for a valid product ID, non-negative price, and non-negative quantity.
4. If the product ID is empty or the price or quantity is negative, display **"INVALID RECORD"**.
5. For valid records, calculate the inventory value by multiplying the price by the quantity.
6. Display the product ID and its inventory value.
7. Check whether the stock quantity is less than 10.
8. If the quantity is less than 10, display a message recommending replenishment.
9. Repeat the process for all product records.
10. Stop.

## 5.FLOWCHART:




## 6.PSEUDO CODE:

```
BEGIN

    SET reorder_level = 10

    FOR each product in products

        READ product_id, price, quantity

        IF product_id is empty OR price < 0 OR quantity < 0 THEN
            DISPLAY "INVALID RECORD"
```

```
        ELSE
            inventory_value = price * quantity
            DISPLAY product_id, inventory_value

            IF quantity < reorder_level THEN
                DISPLAY "Requires replenishment"
            ENDIF

        ENDIF

    ENDFOR

END
```

## 7.PYTHON SOURCE CODE:

```
products = [
    ("P101", 25.0, 30),
    ("P102", 120.0, 6),
    ("P103", -5.0, 12),
    ("P104", 50.0, 0)
]

reorder_level = 10

for product in products:
    product_id, price, quantity = product

    if product_id == "" or price < 0 or quantity < 0:
        print(product_id, "INVALID RECORD")
        continue
```

```
    value = price * quantity
    print(product_id, "inventory value:", round(value, 2))

    if quantity < reorder_level:
        print(product_id, "requires replenishment")
```

## 8.TESTCASES:

**Test Case 1:** When the product ID is P101, the price is 25.0, and the quantity is 30, the program calculates and displays the inventory value as 750.0.

**Test Case 2:** When the product ID is P102, the price is 120.0, and the quantity is 6, the program displays the inventory value as 720.0 and recommends replenishment because the quantity is below 10.

**Test Case 3:** When the product ID is P103, the price is negative (-5.0), and the quantity is 12, the program displays "INVALID RECORD".

**Test Case 4:** When the product ID is P104, the price is 50.0, and the quantity is 0, the program displays the inventory value as 0.0 and recommends replenishment.

**Test Case 5:** When the product ID is empty, the program displays "INVALID RECORD".

## 9.CODE AND OUTPUT SCREENSHOTS:

```
products = [
    ("P101", 25.0, 30),
    ("P102", 120.0, 6),
    ("P103", -5.0, 12),
    ("P104", 50.0, 0)
]

reorder_level = 10

for product in products:
    product_id, price, quantity = product

    if product_id == "" or price < 0 or quantity < 0:
        print(product_id, "INVALID RECORD")
        continue

    value = price * quantity
    print(product_id, "inventory value:", round(value, 2))

    if quantity < reorder_level:
        print(product_id, "requires replenishment")
```

OUTPUT:

```
P101 inventory value: 750.0
P102 inventory value: 720.0
P102 requires replenishment
P103 INVALID RECORD
P104 inventory value: 0.0
P104 requires replenishment
```

## 10.CONCLUSION:

The Tuple-Based Product Catalogue Audit program successfully validates product records, calculates inventory values, and identifies products that require replenishment. It detects invalid records with negative prices, negative quantities, or empty product IDs. Thus, the program helps maintain accurate inventory records and manage product stock efficiently.

## CASE 50: Tuple-Based Route Distance Analysis

### 1.PROBLEM STATEMENT:

Store geographical coordinates as tuples, calculate distances between successive points using a supplied formula, and identify the longest route segment.

### 2.OBJECTIVES:

1. Store route coordinates using tuples.
2. Calculate the distance between successive points.
3. Use loops and mathematical operations to calculate distances.
4. Identify the longest segment of the route.

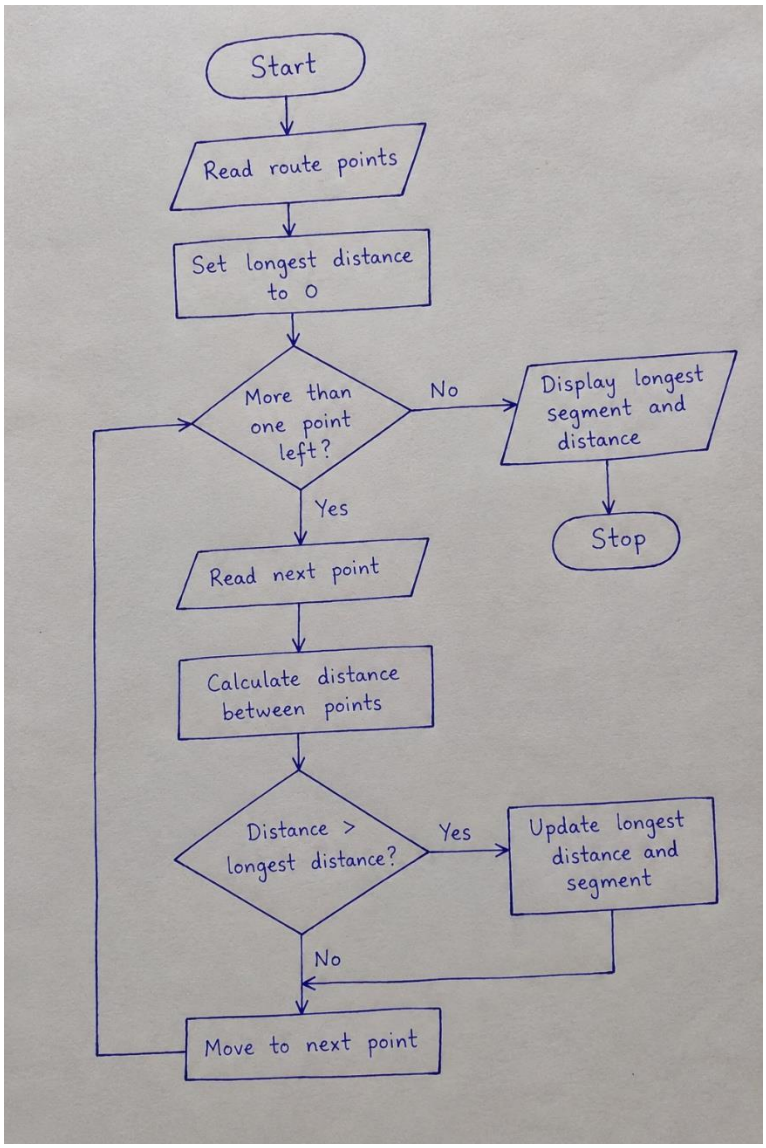
### 3.ASSUMPTIONS:

1. Each point is represented as a tuple (x, y).
2. All coordinates use the same units.
3. Distance is calculated using the Euclidean distance formula: $\sqrt{((x_2 - x_1)^2 + (y_2 - y_1)^2)}$.

### 4.ALGORITHMS:

1. Start.
2. Create a list of coordinate tuples representing route points.
3. Compare each point with the next point.
4. Calculate the horizontal and vertical differences between the two points.
5. Calculate the Euclidean distance between the successive points.
6. Compare the calculated distance with the longest distance found so far.
7. If the calculated distance is greater, update the longest distance and save the corresponding segment.
8. Repeat the process for all successive point pairs.
9. Display the longest route segment and its distance.
10. Stop.

## 5.FLOWCHART:




## 6.PSEUDO CODE:

BEGIN

SET longest_distance = -1

FOR each successive pair of points

Calculate dx = x2 - x1

Calculate dy = y2 - y1

```
        distance = square root of (dx * dx + dy * dy)

        DISPLAY distance

        IF distance > longest_distance THEN
            longest_distance = distance
            Save the current segment
        ENDIF

    ENDFOR

    DISPLAY longest segment
    DISPLAY longest distance

END
```

## 7.PYTHON SOURCE CODE:

```
from math import sqrt

points = [
    (0, 0),
    (3, 4),
    (9, 4),
    (10, 12)
]

longest_distance = -1
longest_segment = None

for i in range(len(points) - 1):
```

```
    x1, y1 = points[i]
    x2, y2 = points[i + 1]

    dx = x2 - x1
    dy = y2 - y1

    distance = sqrt(dx ** 2 + dy ** 2)

    print("Segment", points[i], "to", points[i + 1],
          "distance:", round(distance, 2))

    if distance > longest_distance:
        longest_distance = distance
        longest_segment = (points[i], points[i + 1])

print("Longest segment:", longest_segment)
print("Longest distance:", round(longest_distance, 2))
```

## 8.TESTCASES:

**Test Case 1:** When two points are identical, such as (2, 3) and (2, 3), the distance is calculated as 0.0.

**Test Case 2:** When the points are (0, 0) and (3, 4), the program calculates the distance as 5.0.

**Test Case 3:** When the route contains multiple segments, the program calculates the distance between each successive pair of points.

**Test Case 4:** When the route points are (0, 0), (3, 4), (9, 4), and (10, 12), the program identifies the longest segment as ((9, 4), (10, 12)) with a distance of 8.06.

## 9.CODE AND OUTPUT SCREENSHOTS:

```
CASE50.py > ...
from math import sqrt

points = [
    (0, 0),
    (3, 4),
    (9, 4),
    (10, 12)
]

longest_distance = -1
longest_segment = None

for i in range(len(points) - 1):
    x1, y1 = points[i]
    x2, y2 = points[i + 1]

    dx = x2 - x1
    dy = y2 - y1

    distance = sqrt(dx ** 2 + dy ** 2)

    print("Segment", points[i], "to", points[i + 1],
          "distance:", round(distance, 2))

    if distance > longest_distance:
        longest_distance = distance
        longest_segment = (points[i], points[i + 1])

print("Longest segment:", longest_segment)
print("Longest distance:", round(longest_distance, 2))
```

OUTPUT:

```
Segment (0, 0) to (3, 4) distance: 5.0
Segment (3, 4) to (9, 4) distance: 6.0
Segment (9, 4) to (10, 12) distance: 8.06
Longest segment: ((9, 4), (10, 12))
Longest distance: 8.06
```

## 10.CONCLUSION:

The Tuple-Based Route Distance Analysis program successfully calculates the distance between successive route points using the Euclidean distance formula. It identifies and displays the longest segment along the route. Thus, the program demonstrates the use of tuples, loops, and mathematical operations to analyze route distances efficiently.